Graph Traversal and Connectivity

Peerapon S.

CPE 231 Algorithms

Topics

Graph review

Topological sorting

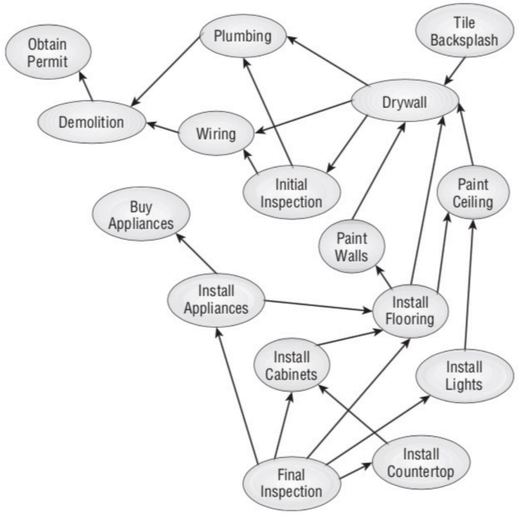
Critical path

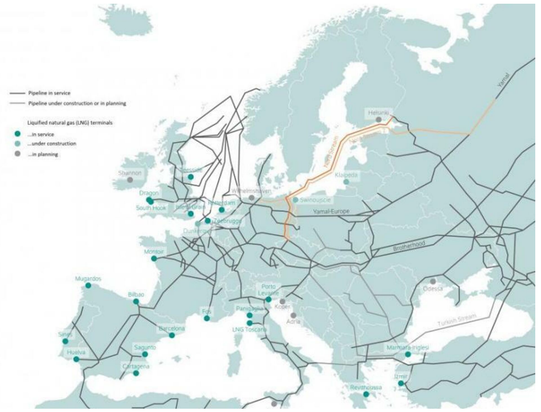
Graph connectivity

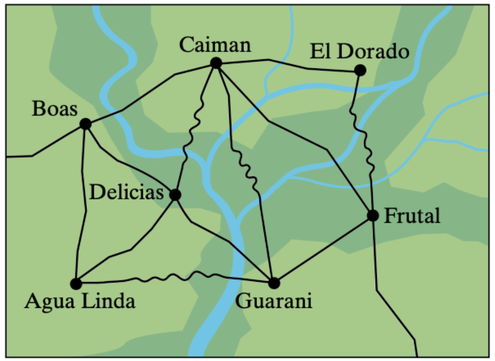
- Depth first search (DFS) review
- Articulation points
- Strongly connected components

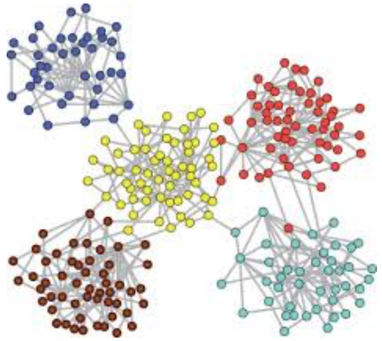
Graph Applications

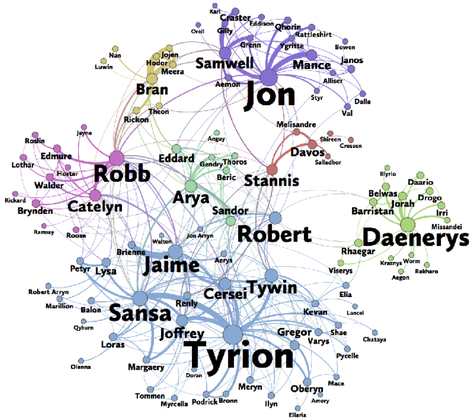
Task conflicts
Critical path
Critical points analysis
Network design
Customer segmentation
Shortest path
etc.



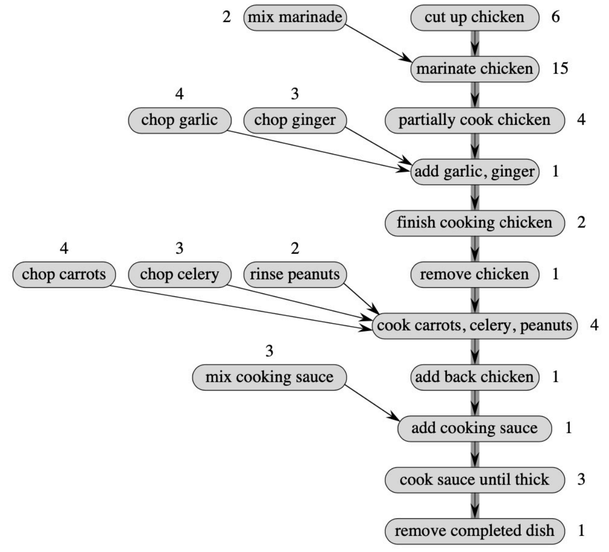








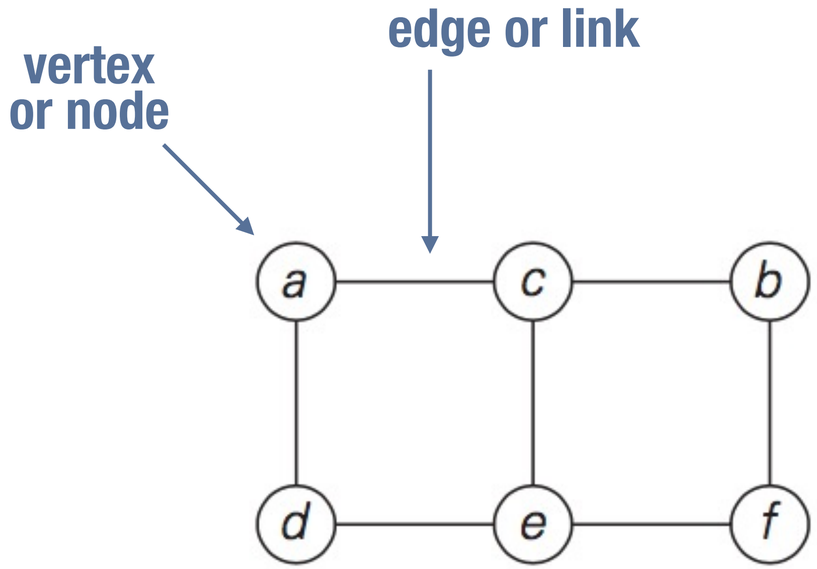




Graph

A graph $\boldsymbol{G} = (\boldsymbol{V}, \boldsymbol{E})$ consists of a *nonempty* set $\boldsymbol{V}$ of **vertices/nodes** $\{v_1, v_2, \ldots, v_N\}$ and a set $\boldsymbol{E}$ of **edges/links** $\{(v_i, v_j)\}$.

$$V = \{a, b, c, d, e, f\}, E = \{(a, c), (a, d), (b, c), (b, f), (c, e), (d, e), (e, f)\}$$



Graph size parameters

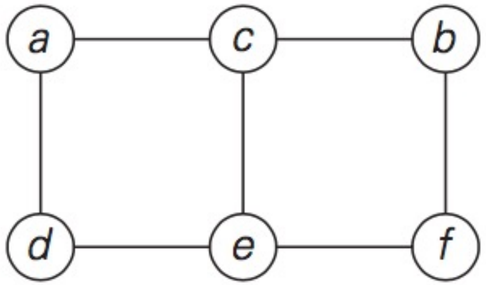
$n = |V| = 6$

$m = |E| = 7$

Two nodes are neighbors if they are connected by edges.

Unigraph and Digraph

Undirected/Uni-directional Graph or Unigraph

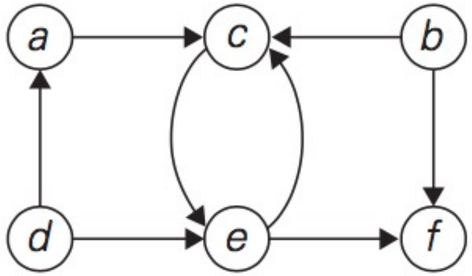


$V = \{a, b, c, d, e, f\}$

$E = \{(a, c), (a, d), (b, c), \ldots, (e, f)\}$

(c, e) is the same as (e, c)

Directed Graph or Digraph

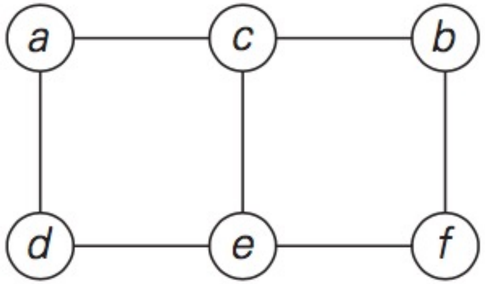


$V = \{a, b, c, d, e, f\}$

$E = \{(a, c), (a, d), (b, c), \ldots, (c, e), (e, c), (e, f)\}$

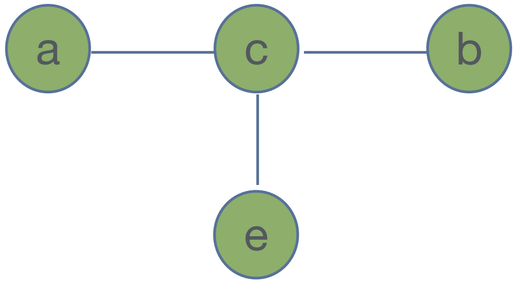
(c, e) not the same as (e, c)

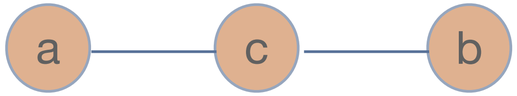
Subgraph



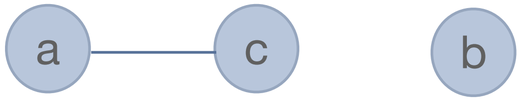
$$G = (V, E)$$
$$V = \{a, b, c, d, e, f\}$$
$$E = \{(a, c), (a, d), (b, c), \ldots, (e, f)\}$$

Possible subgraphs of G







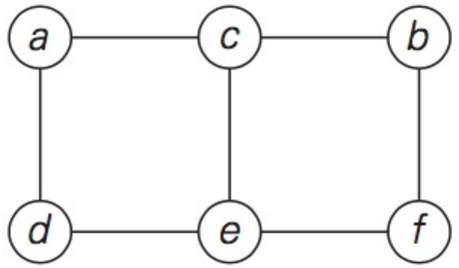




Graph Representations

Adjacency matrix and ***Adjacency list***

When to use which ?



$$\boldsymbol{A} = \begin{array}{c} \\ a \\ b \\ c \\ d \\ e \\ f \end{array} \begin{array}{c} \begin{array}{cccccc} a & b & c & d & e & f \end{array} \\ \begin{bmatrix} 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 \end{bmatrix} \end{array}$$

a	→	c	→	d		
b	→	c	→	f		
c	→	a	→	b	→	e
d	→	a	→	e		
e	→	c	→	d	→	f
f	→	b	→	e		

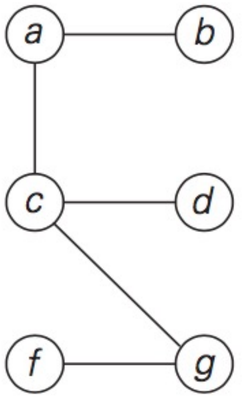
$$\boldsymbol{A}(i,j) = \begin{cases} 1 & \text{if } v_i \text{ is adjacent to } v_j \\ 0 & \text{otherwise} \end{cases}$$

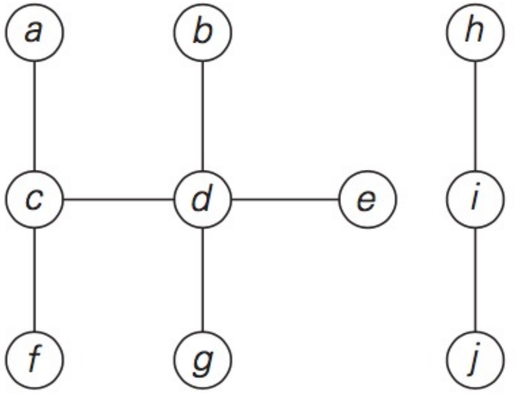
Acyclic Graph

Acyclic graph (no cycle/loop)

- $|E| = |V| - 1$
- Exactly one path between two nodes (unigraph)
- Has a unique root where all other nodes descend.

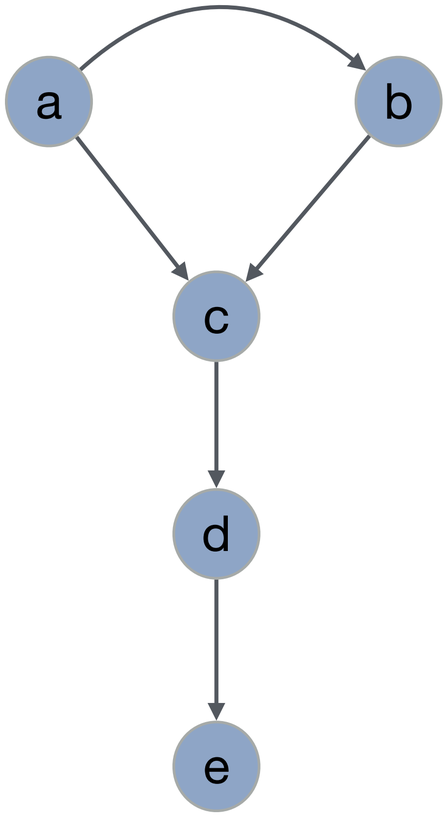
Forest: Acyclic graph that is not connected.



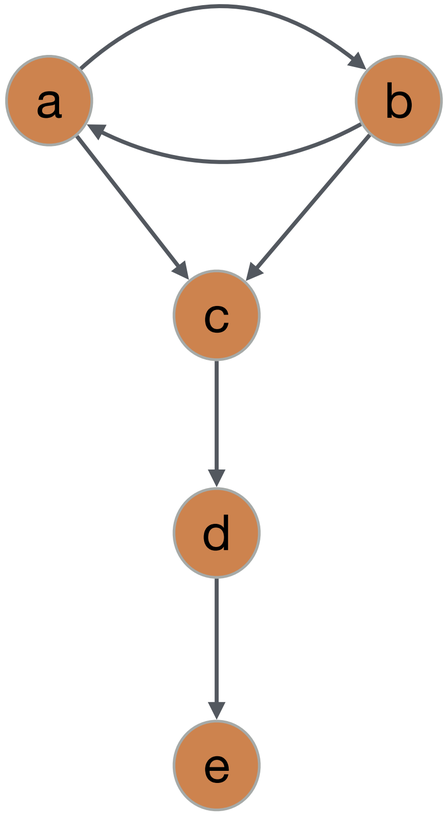


Directed Acyclic Graph (DAG)

Digraph with no cycle/loop



not DAG



Basic Graph Algorithms

Graph traversal -- **Depth-first** and **Breadth-first searches (DFS, BFS)**

- Systematically visit nodes in the graph
- Discover *connectivity, cycles/loops, connected components, critical nodes.*

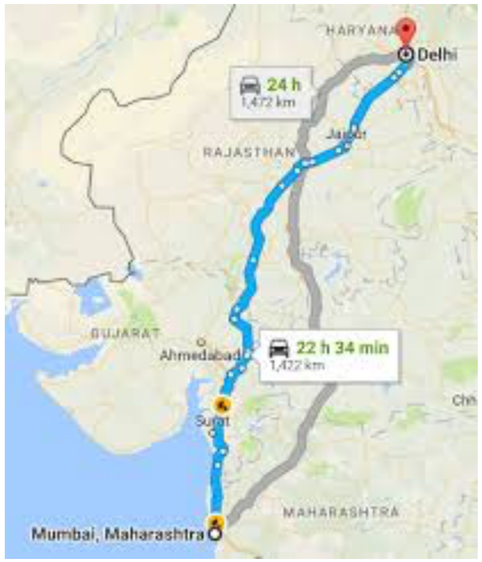


Minimum spanning tree -- **Prim's** and **Kruskal's** algorithms

- Find least cost tree that covers a set of nodes
- Ex: Network design, Subgroups or communities discovery

Shortest path tree -- ***Dijkstra's*** algorithm

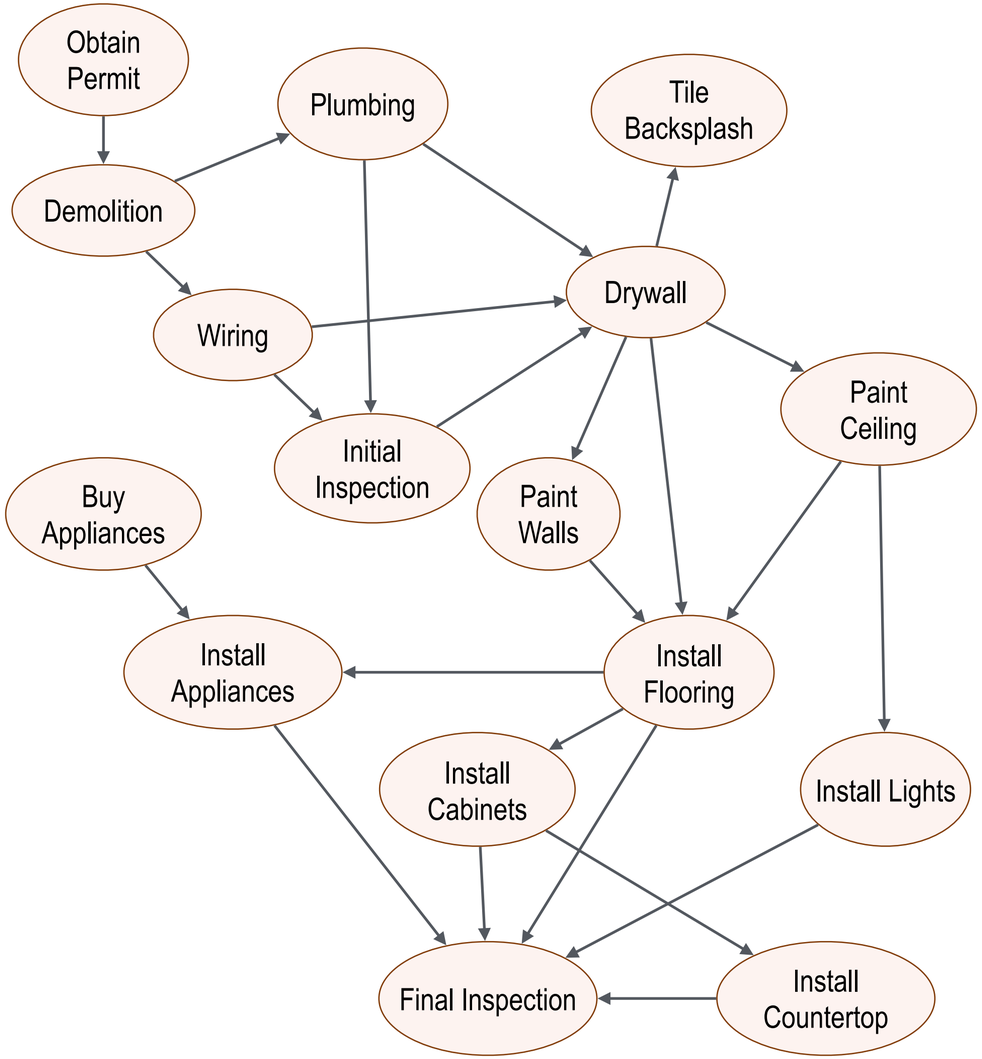
- Find best (least-cost, least-hop) path between two nodes
- Ex: Optimal route, Closedness centrality



Scenario: Kitchen Remodeling

TASK	PREREQUISITE
Obtain permit	—
Buy appliances	—
Install appliances	Buy appliances
Demolition	Obtain permit
Wiring	Demolition
Drywall	Wiring
Plumbing	Demolition
Initial inspection	Wiring
Initial inspection	Plumbing
Drywall	Plumbing
Drywall	Initial inspection
Paint walls	Drywall
Paint ceiling	Drywall
Install flooring	Paint walls
Install flooring	Paint ceiling
Final inspection	Install flooring
Tile backsplash	Drywall
Install lights	Paint ceiling
Final inspection	Install lights
Install cabinets	Install flooring
Final inspection	Install cabinets
Install countertop	Install cabinets
Final inspection	Install countertop
Install flooring	Drywall
Install appliances	Install flooring
Final inspection	Install appliances

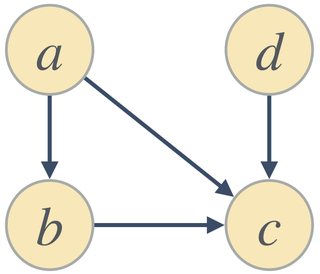
R.Stephens, Essential Algorithms, Wiley 2013



Is there any task order conflict ?
What is the sequence of tasks to perform ?

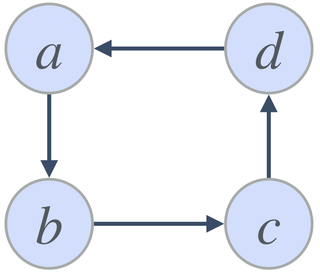
Topological Ordering/Sorting

Linear ordering of vertices in a digraph such that for every directed edge (u, v), u comes before v in the ordering.



$E = \{(a, b), (a, c), (b, c), (d, c)\}$

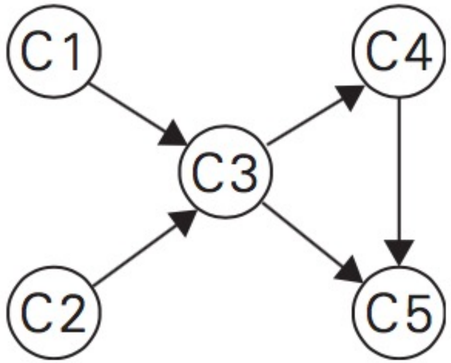
a, b, c, d
a, b, d, c
d, a, b, c



$E = \{(a, b), (b, c), (c, d), (d, a)\}$

a, b, c, d
a, b, d, c
d, a, b, c

Topological Sorting Problem



Five courses with pre-requisite relationships.
What is the order to take the courses?

Find a ***topology ordering*** of a digraph:

Can we list all vertices in a digraph such that u comes before v for every directed edge (u, v) ?

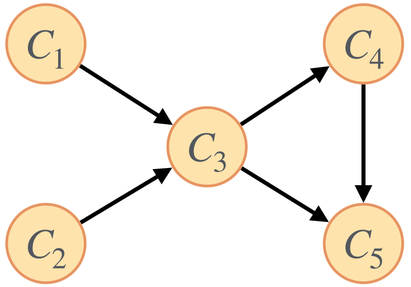
A digraph has a topological only when it is acyclic, i.e., *Directed Acyclic Graph (DAG)*.

Allow for detecting any conflict in schedule, circular dependency loop.

Source Removal (Kahn's) Algorithm

Repeatedly delete a vertex with no incoming edge together with its outgoing edges.

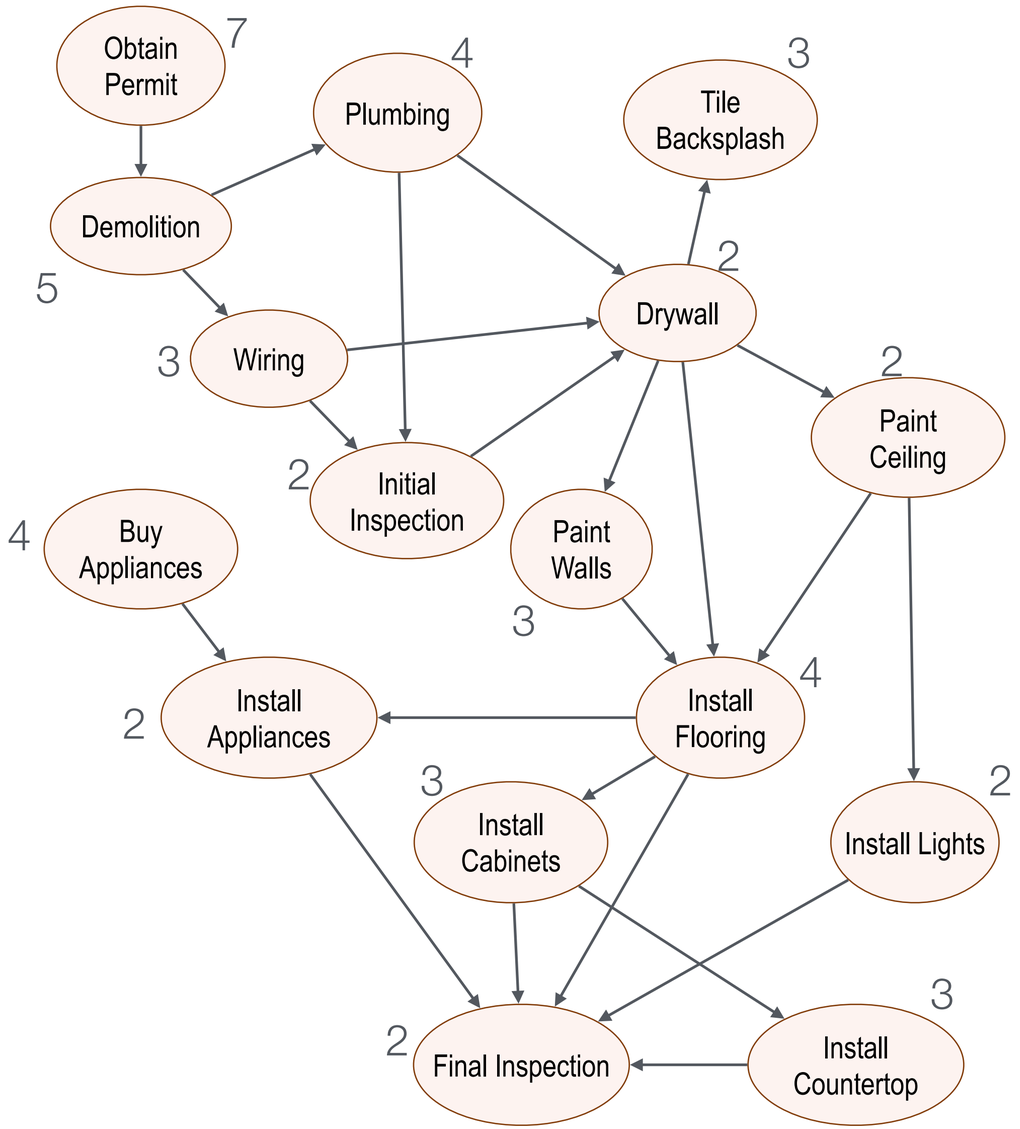
The task sequence is the order of deleted vertices.



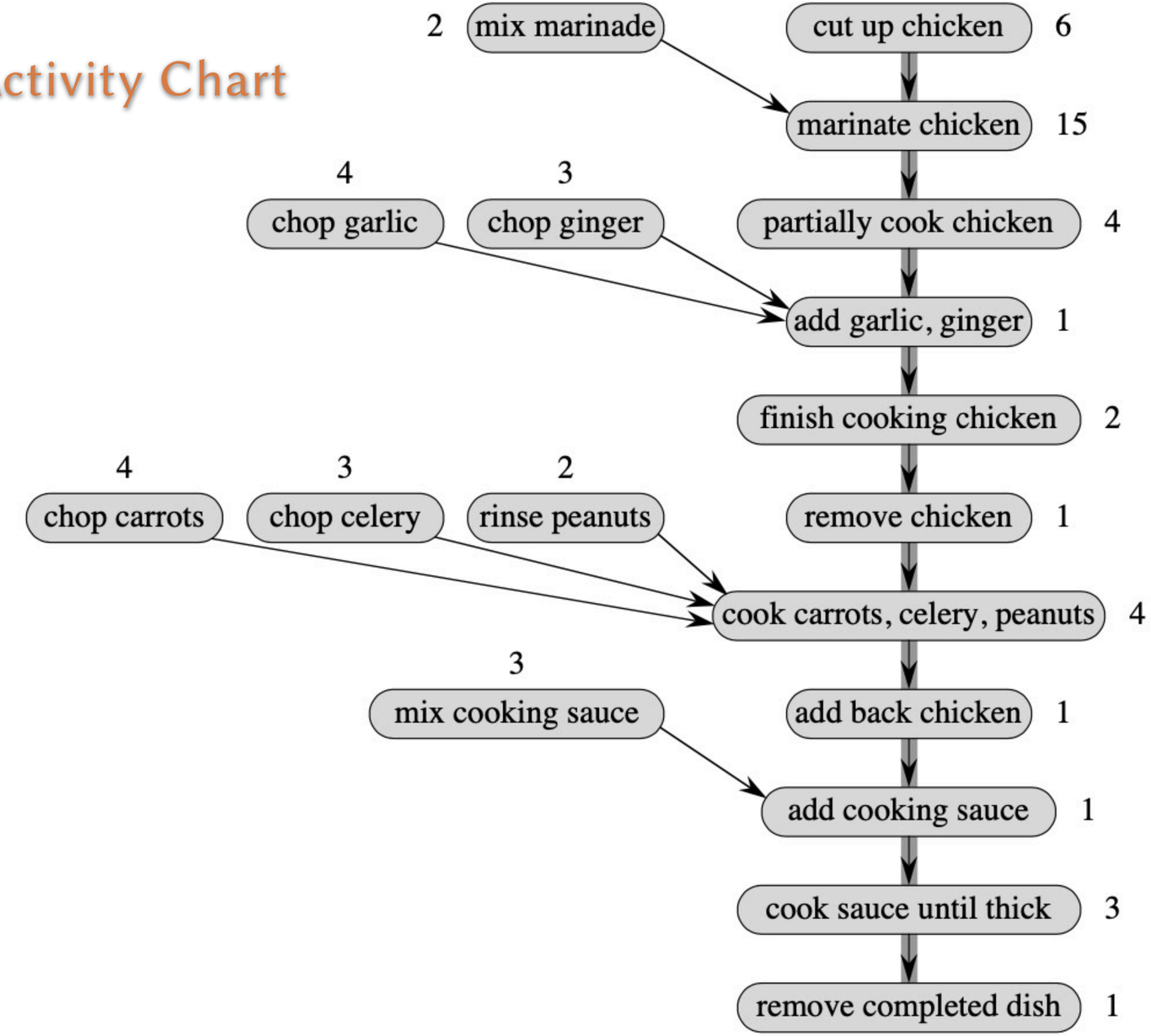
Next Step: Estimating Project Time

Assume that the tasks have no conflict (DAG) and each task time is known.

How to estimate the minimum amount of time to complete the project?



Project Activity Chart





How quick can we finish Kung-Pao chicken ?

Critical Path in Project

$\sum$ task times along the critical path giving the minimum possible time for the entire project.

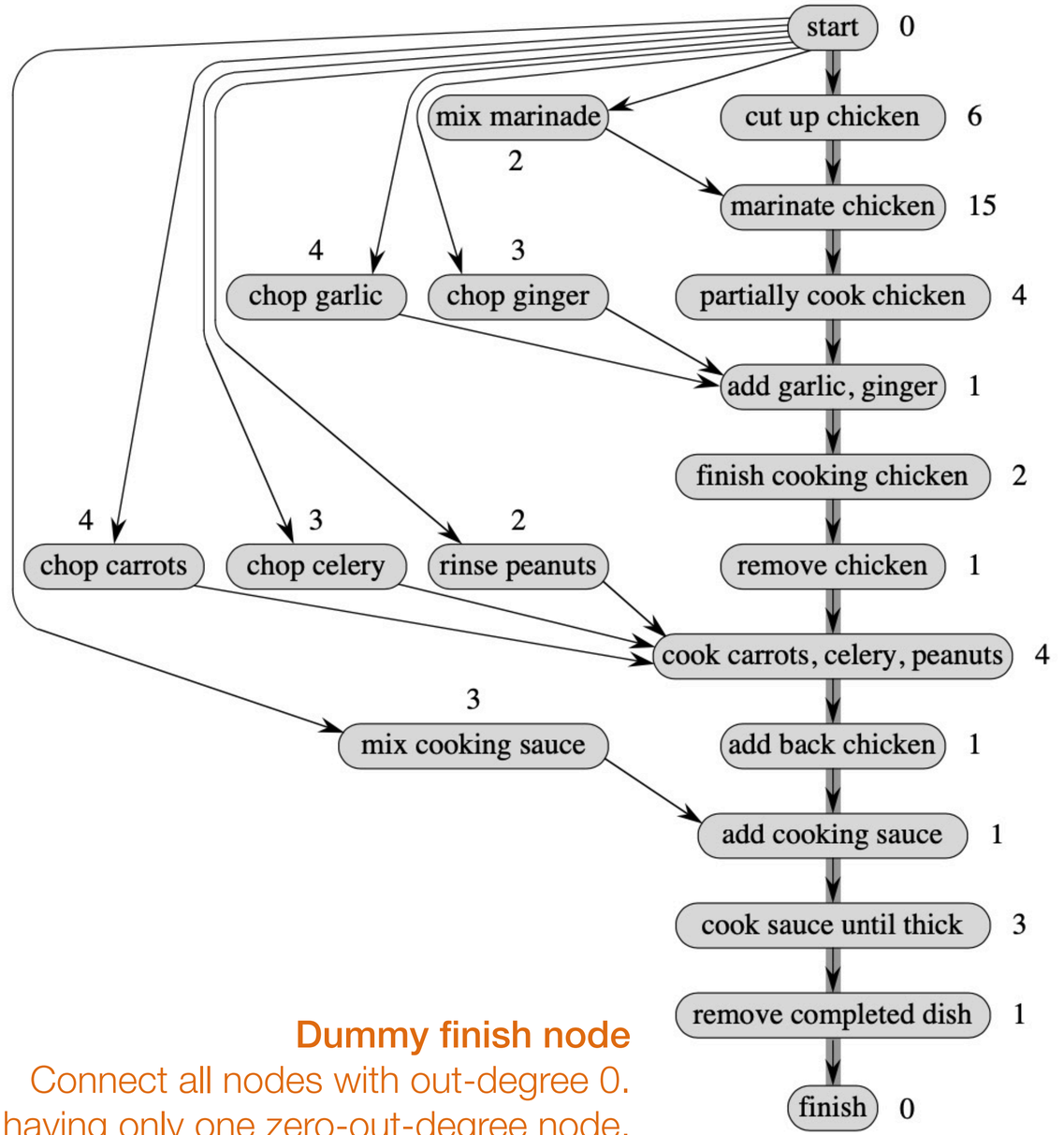
How do we find such critical path ?

- Longest path based on task times, or
- Shortest path based on negated task times

Construct a ***weighted DAG*** from the input.

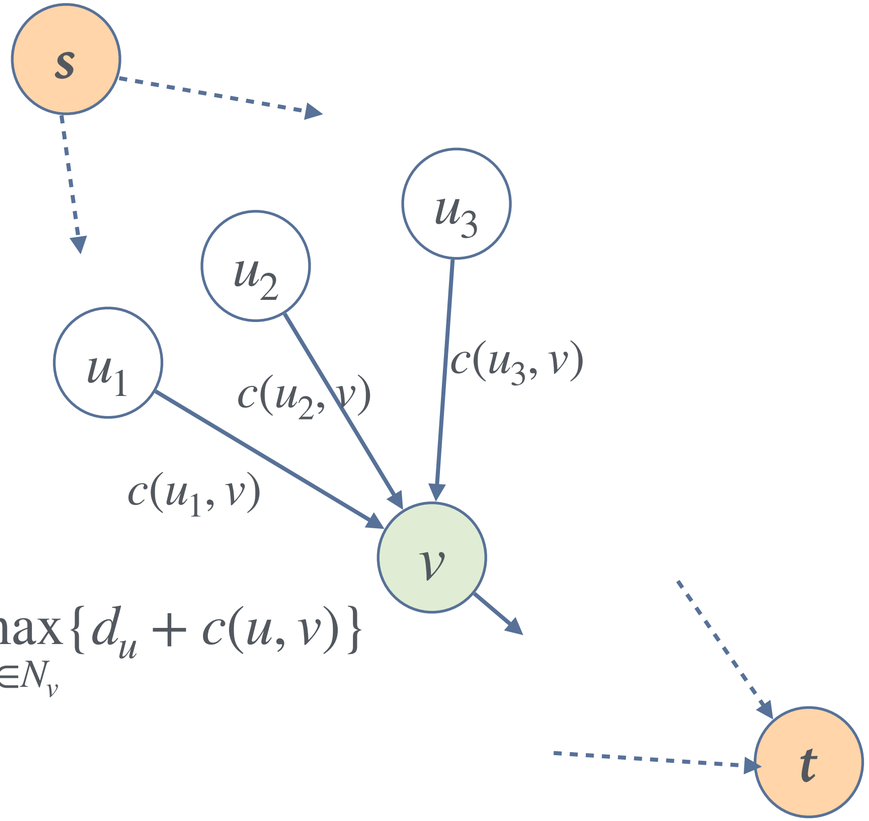
- Add dummy ***start node*** *s*.
- ***Push task times to incoming edge weights.***
- Find the longest path length from *s* in the graph.

Dummy start node
Connect all nodes with in-degree 0



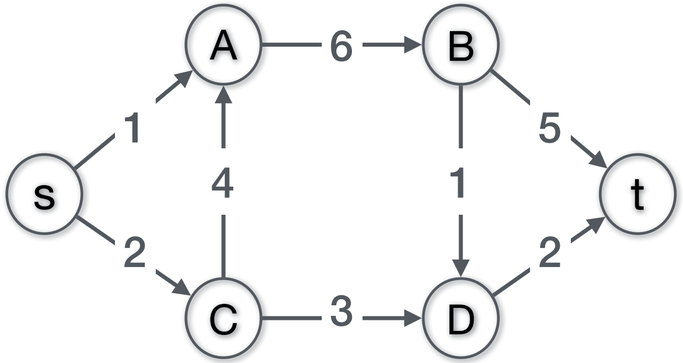
Dummy finish node
Connect all nodes with out-degree 0.
Not needed if having only one zero-out-degree node.

Longest-Path DAG Algorithm



$$d_v = \max_{u \in N_v} \{d_u + c(u, v)\}$$

Keep the previous node in each step to derive the critical path.



Topological sort (Linearized) order: s, C, A, B, D, t

$$d_S = 0$$

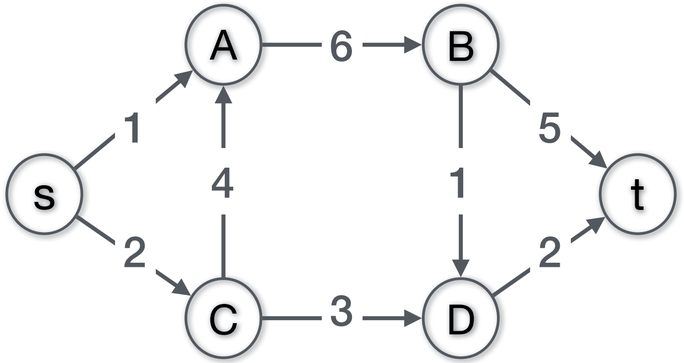
$$d_C =$$

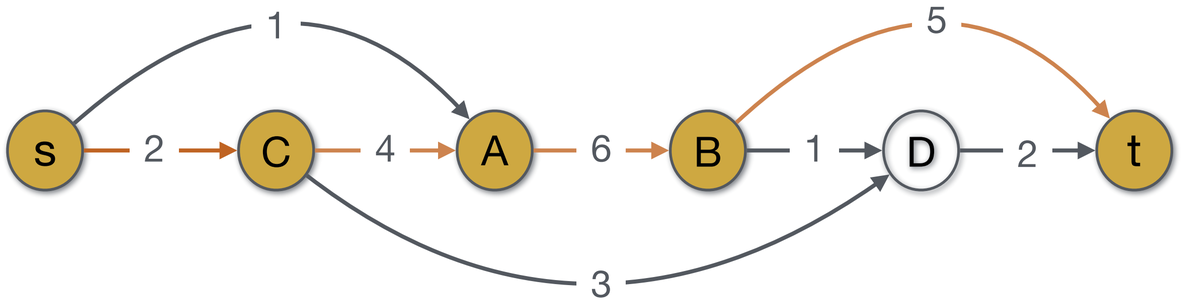
$$d_A =$$

$$d_B =$$

$$d_D =$$

$$d_t =$$





Algorithm Longest-Path-DAG(G)

Input: Weighted DAG $G = (V, E)$

Output: Largest path cost in G from the 1st vertex in the linearized order.

Topological sort G to get the linearized order

$d_v \leftarrow 0, \forall v \in V$

for each vertex $v \in V$ in linearized order **do**

    $d_v \leftarrow \max_{(u,v)\in E}\{d_u + c(u, v)\}$

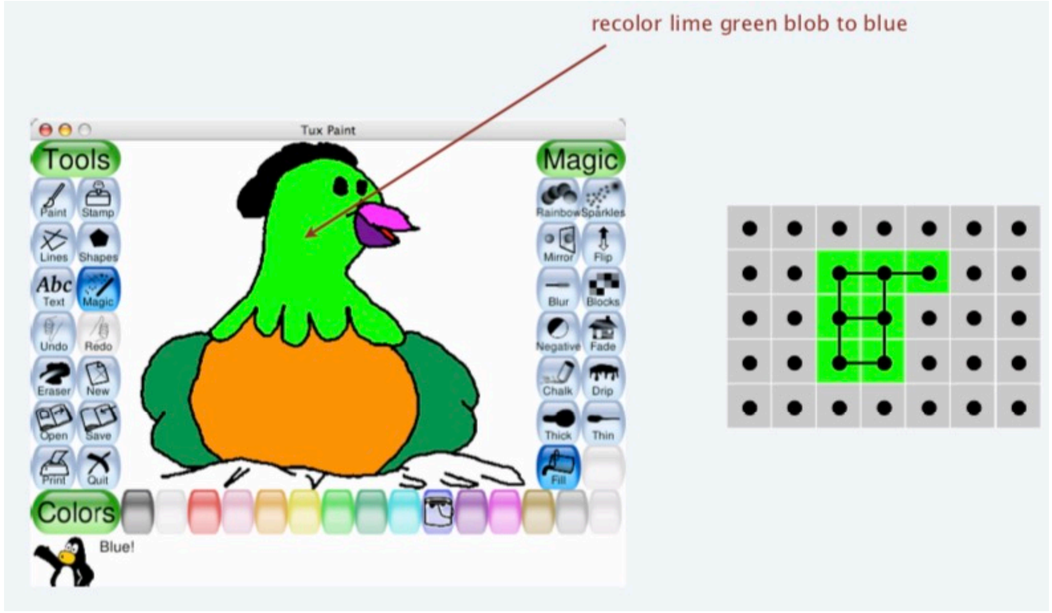
    Update prev$[v]$

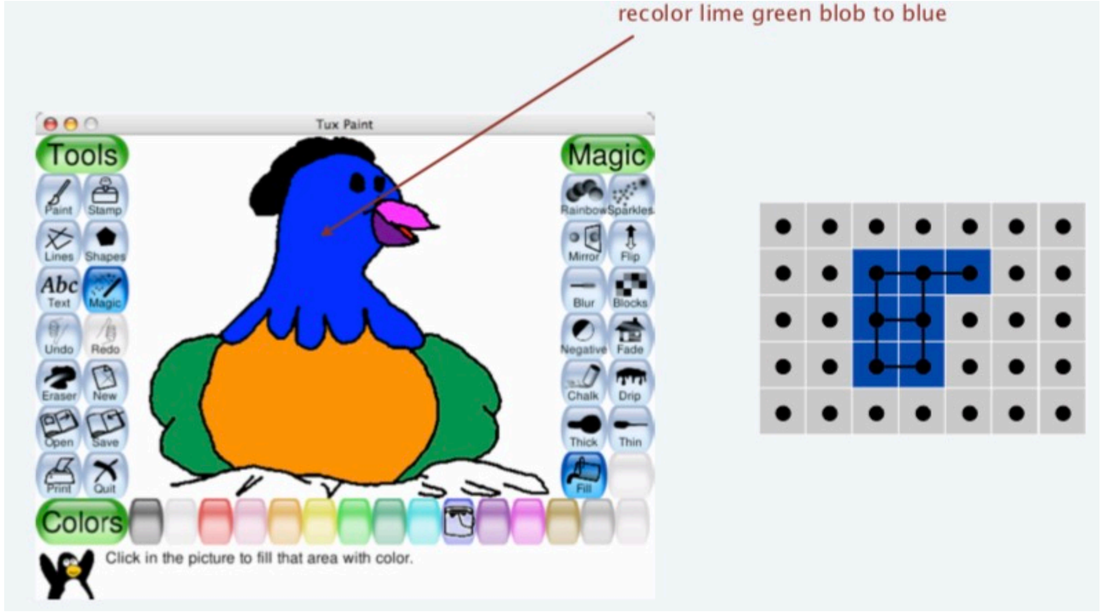
return $\max_{v\in V} d_v$

Self Test: Longest-Path DAG

https://forms.cloud.microsoft/r/s0V7GEc4DZ?origin=lprLink

Scenario: Flood Fill





Graph Connectivity

Def: A ***graph*** is ***connected*** if there is a path for every node pairs.

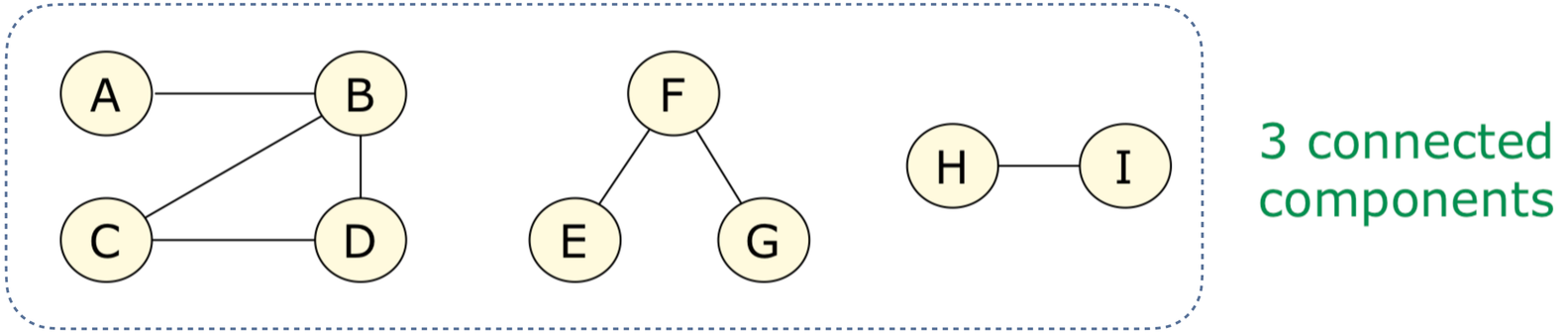
Def: A ***digraph*** is ***strongly connected*** if there is a path for every node pairs.

A ***connected component*** in a graph is a ***subgraph*** where:

- A path exists between every node pairs in the subgraph.
- The subgraph is not part of a larger subgraph with the above property.

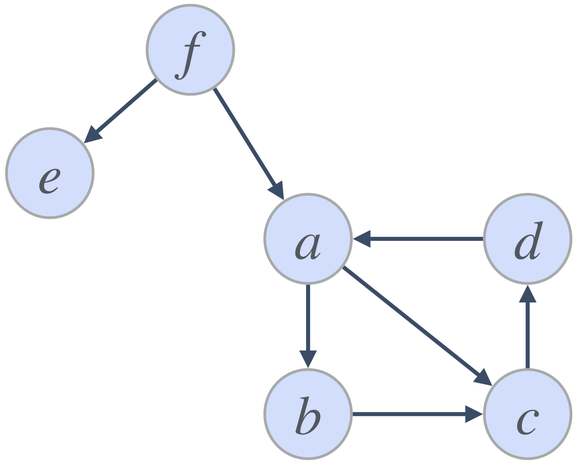
Use depth-first search to systematically list vertices and determine connected components.

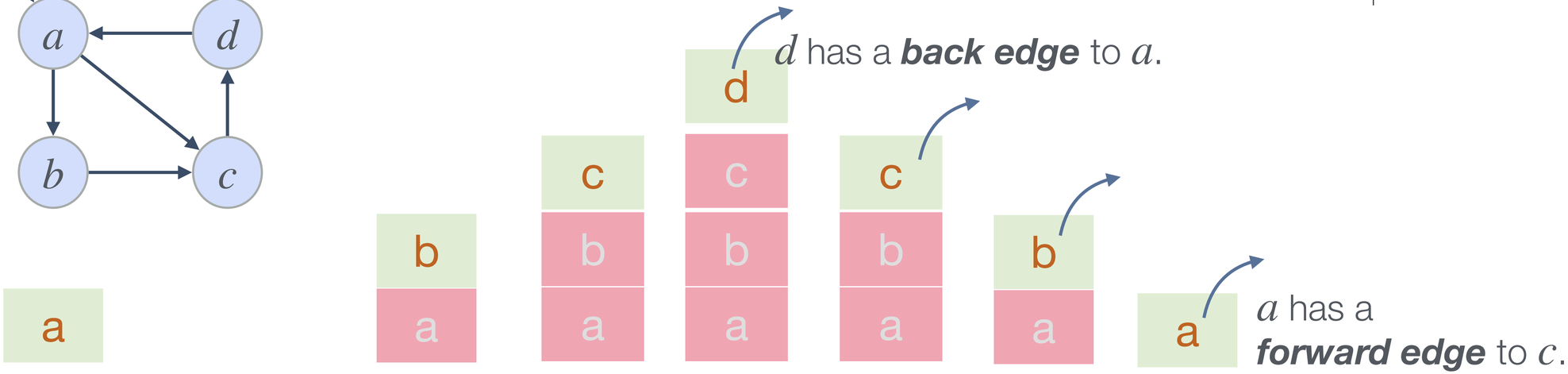
$$V = \{A, B, C, D, E, F, G, H, I\}$$

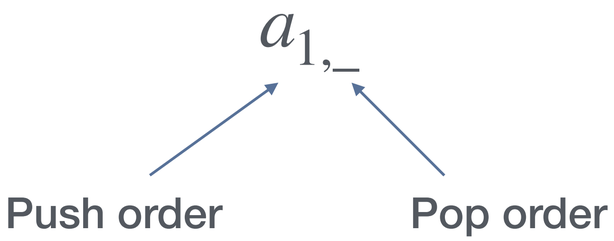


Review: Tracking DFS Operation by using Stack

	a	b	c	d	e	f
a	0	1	1	0	0	0
b	0	0	1	0	0	0
c	0	0	0	1	0	0
d	1	0	0	0	0	0
e	0	0	0	0	0	0
f	1	0	0	0	1	0



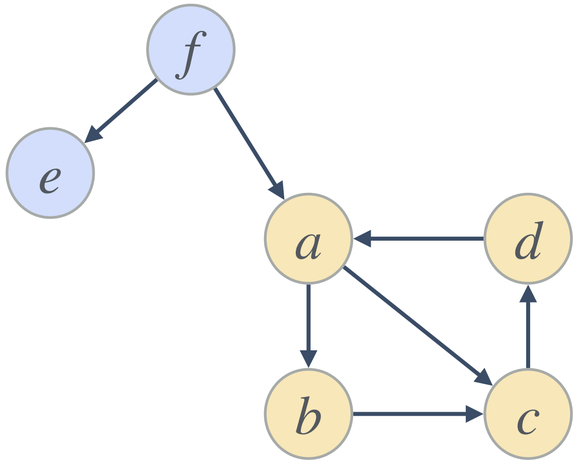




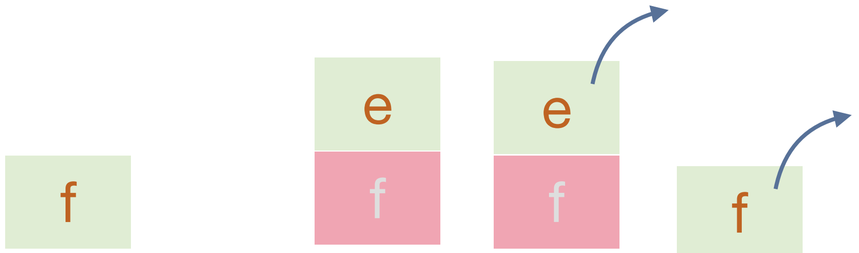
		$d_{4,1}$	$d_{4,1}$	$d_{4,1}$	$d_{4,1}$
	$c_{3,_}$	$c_{3,_}$	$c_{3,2}$	$c_{3,2}$	$c_{3,2}$
$b_{2,_}$	$b_{2,_}$	$b_{2,_}$	$b_{2,_}$	$b_{2,3}$	$b_{2,3}$
$a_{1,_}$	$a_{1,_}$	$a_{1,_}$	$a_{1,_}$	$a_{1,_}$	$a_{1,4}$

2nd traversal round

	a	b	c	d	e	f
a	0	1	1	0	0	0
b	0	0	1	0	0	0
c	0	0	0	1	0	0
d	1	0	0	0	0	0
e	0	0	0	0	0	0
f	1	0	0	0	1	0



f has a ***cross edge*** to a.



$f_{5,_}$

$e_{6,_}$ $e_{6,5}$ $e_{6,5}$

$f_{5,_}$ $f_{5,_}$ $f_{5,6}$

Push order : a, b, c, d, f, e

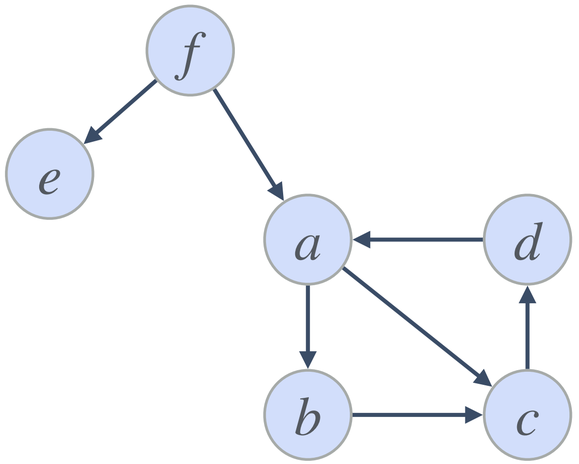
Pop order: d, c, b, a, e, f

DFS stack operation

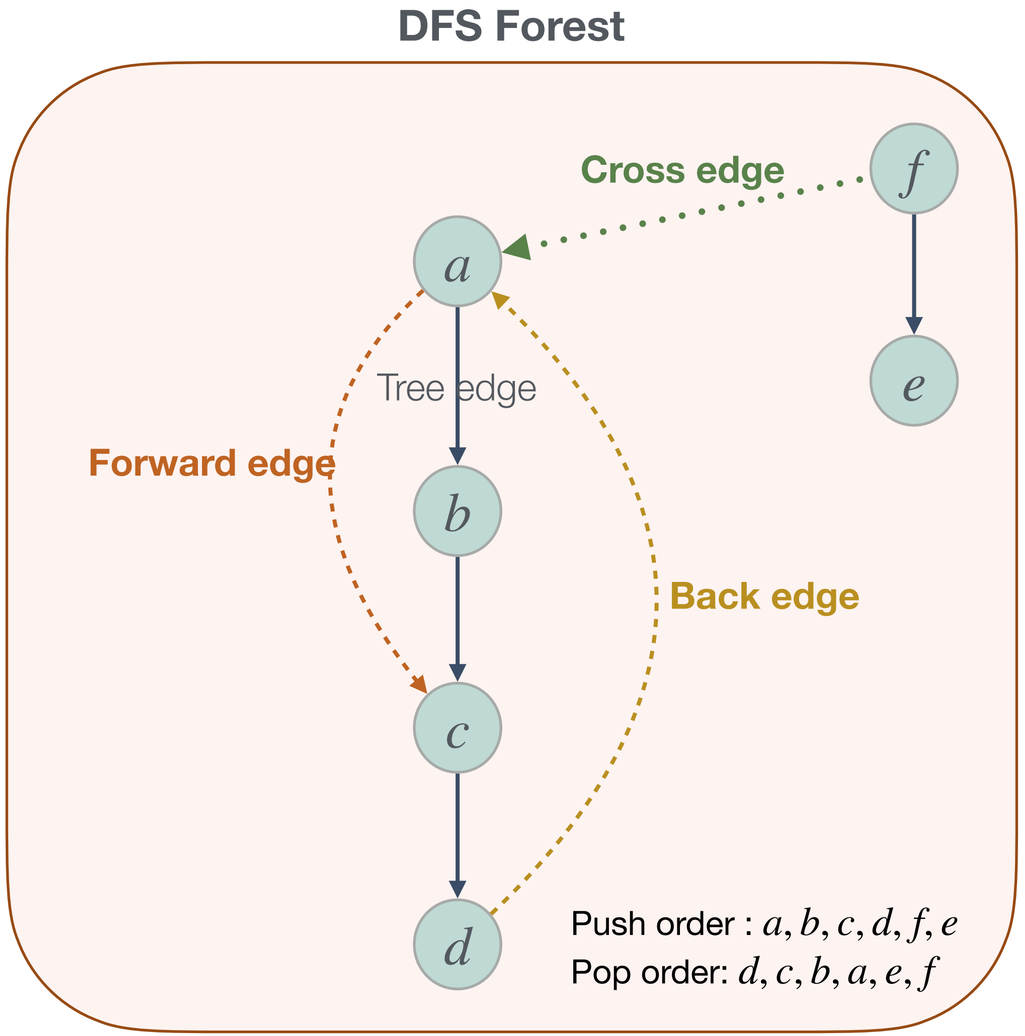
- Push a vertex into the stack when it is visited for the first time, and then its unvisited neighbor.
- Pop off a vertex when it becomes the **dead end**.

Lead to ***two distinct vertex orderings*** (Push or Pop) and ***three additional edge types besides tree edges***

DFS forest to visualize connected components and sequence of nodes processed.



$d_{4,1}$
$c_{3,2}$
$b_{2,3}$ $e_{6,5}$
$a_{1,4}$ $f_{5,6}$



Algorithm DFSTraversal

```
1:  Input: Graph G = ⟨V, E⟩
2:  Output: Visit (push) order Visit[] and finished (pop) order Finish[] of vertices,
    and the number of connected components.
3:
4:  Mark all vertices with 0 (being unvisited)
5:  pushCount, popCount, numComp, Visit[], Finish[] ← 0
6:  for each vertex v in V do
7:      if v is marked with 0 then
8:          DFS(v)
9:          numComp ← numComp+1
10: Return Visit[], Finish[], numComp
11:
12: procedure DFS(v)
13:     pushCount ← pushCount+1
14:     Visit[pushCount] ← v                          ▷ Keep visit order of v
15:     Mark v with pushCount
16:     for each vertex w in V adjacent to v do   ▷ DFS on unvisited neighbor w
17:         if w is marked with 0 then
18:             DFS(w)
19:     popCount ← popCount+1
20:     Finish[popCount] ← v                          ▷ Keep finish order of v
```

$$\boldsymbol{A} = \begin{array}{c} \\ a \\ b \\ c \\ d \\ e \\ f \end{array} \begin{array}{c} \begin{array}{cccccc} a & b & c & d & e & f \end{array} \\ \begin{bmatrix} 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 \end{bmatrix} \end{array}$$

a	→	*c*	→	*d*		
b	→	*c*	→	*f*		
c	→	*a*	→	*b*	→	*e*
d	→	*a*	→	*e*		
e	→	*c*	→	*d*	→	*f*
f	→	*b*	→	*e*		

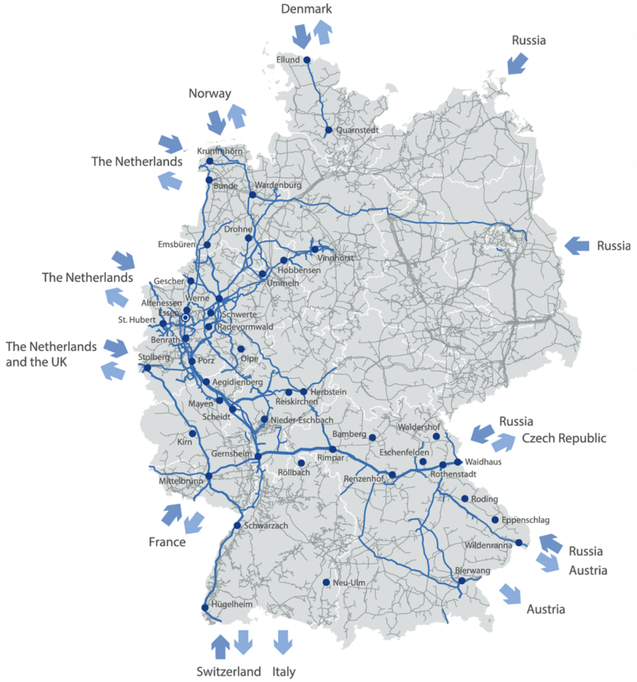
Self Test: DFS Traversal

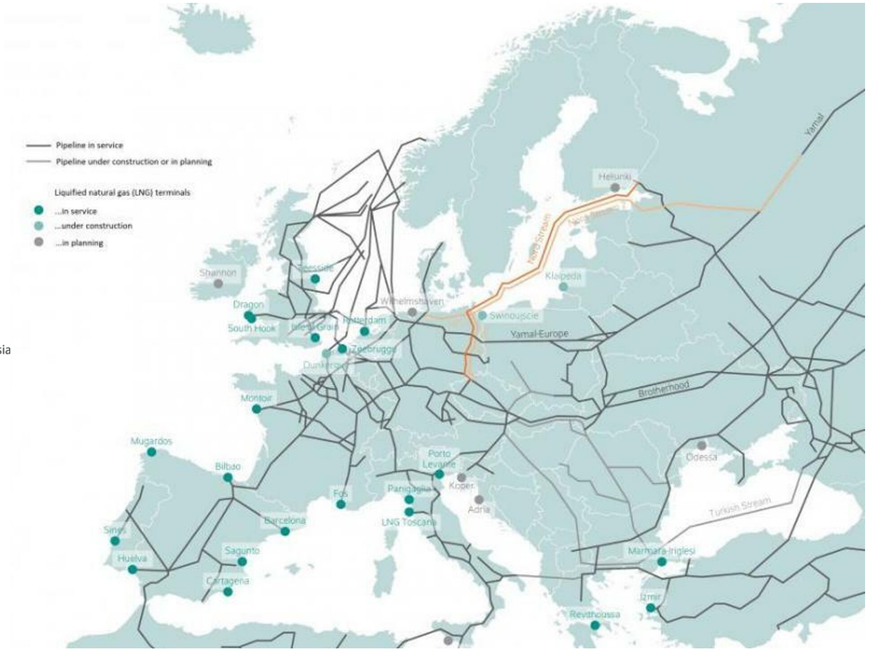


https://forms.office.com/r/kFU0urwsKc?origin=lprLink

Is there any single point of failures in transmission/road/power grid networks ?
What could be potential targets of attack for immediate damage to networks ?



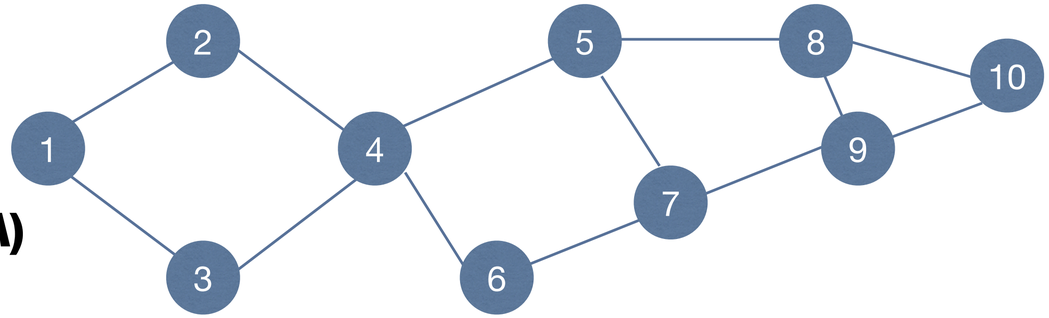




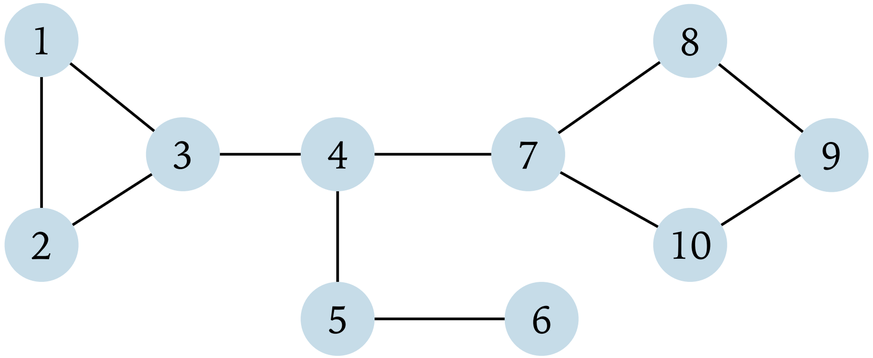
Articulation Point (AP)

Vertex that, if removed, will break a connected graph into many subgraphs or increases the number of connected components in the graph.

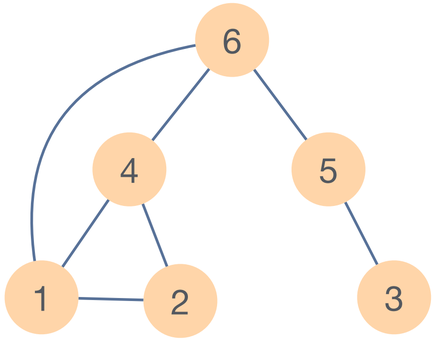
(A)



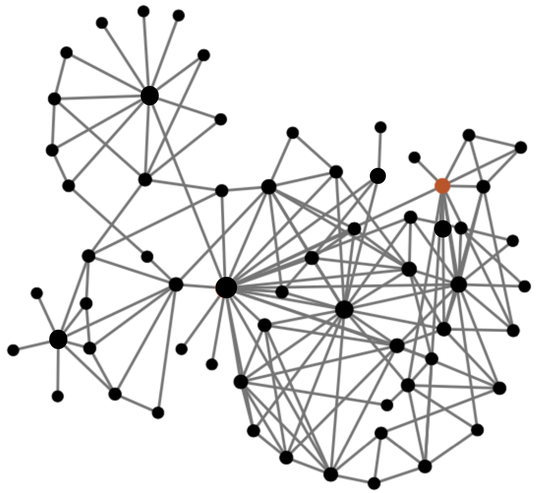
(B)



(C)



(D)



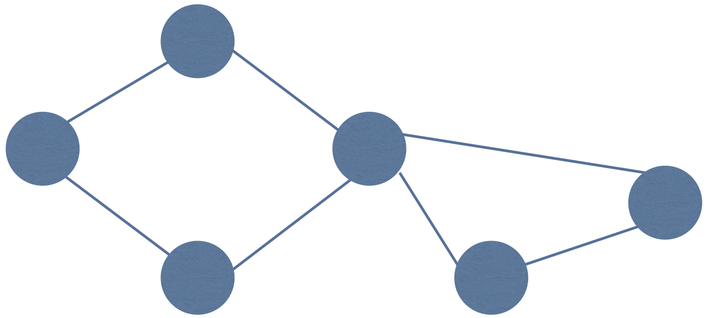
Which nodes are articulation points in these graphs ?

Finding Articulation Points -- Brute-Force Approach

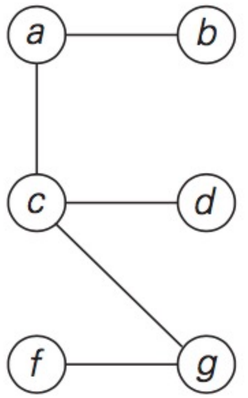
Algorithm BruteForceAP

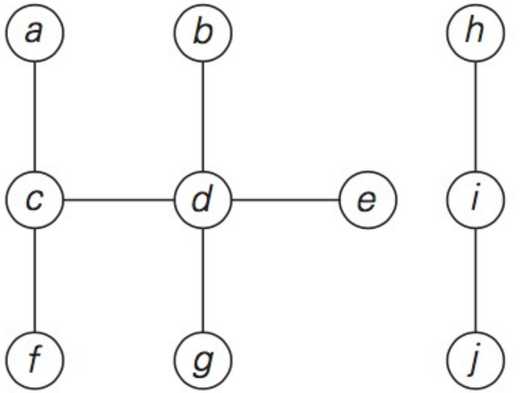
Input: Graph G
Output: Set of articulation points V_A
$N_c \leftarrow$ Number of connected components in G (Run DFS once)
$V_A \leftarrow \phi$
for every vertex $v \in G$ **do**
    $G' \leftarrow G$ with v and its connected edges removed
    Run DFS on G' to determine the number of connected components N_c'
    if $N_c' > N_c$ **then**
        Add v to V_A

return V_A



Tree Review





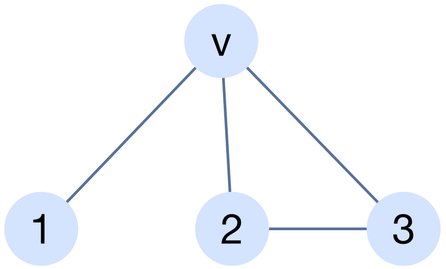
Parent of v: Node u whose edge (u, v) is on the path from the root.

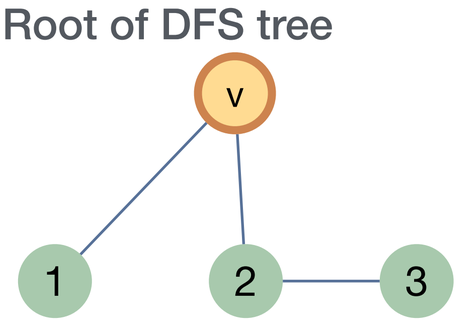
(Proper) Ancestors of v: All nodes from the path from the root to v, (excluding) including node v itself.

(Proper) Descendants of v: All nodes for which v is their ancestor, (excluding) including node v itself.

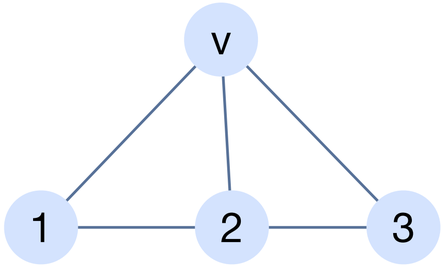
Condition I for Articulation Points

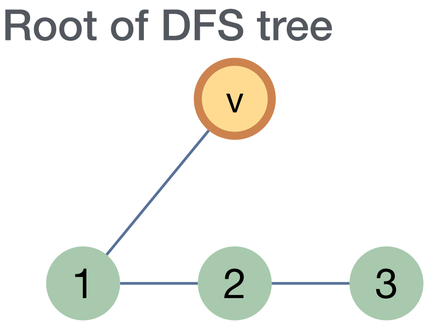
v is a ***root of DFS tree*** and has at least two children.





Is v an AP?

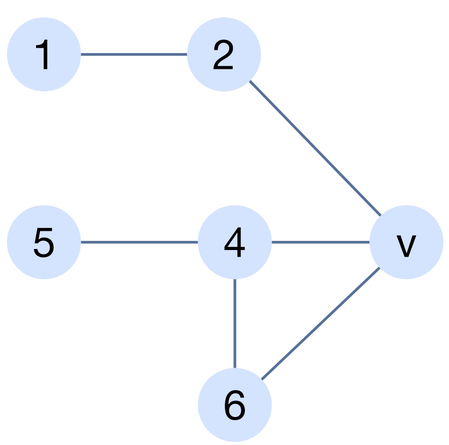


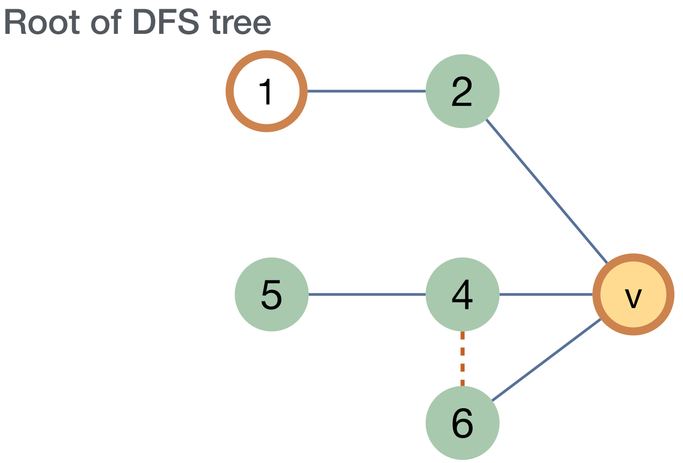


Is v an AP?

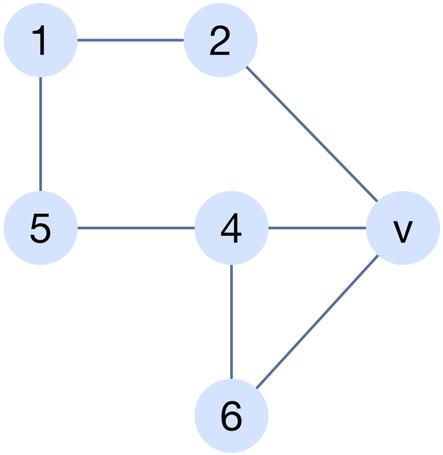
Condition II for Articulation Points

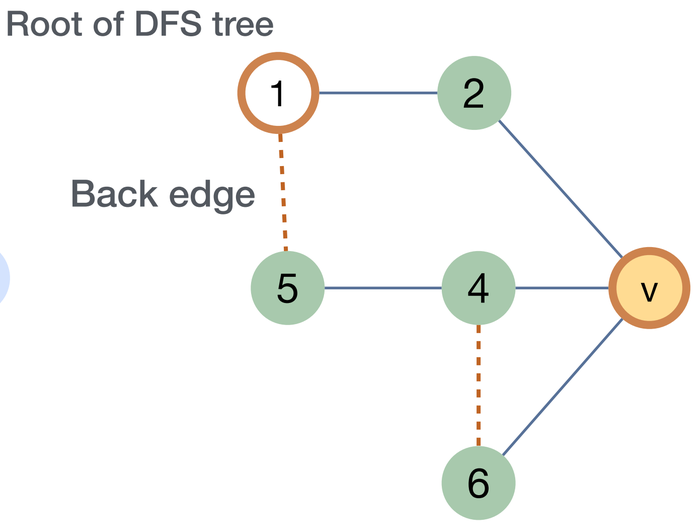
v is an ***internal node*** in DFS tree, and no proper descendants of *v* has back edges to a proper ancestor of *v*.





Is *v* is an articulation point ?



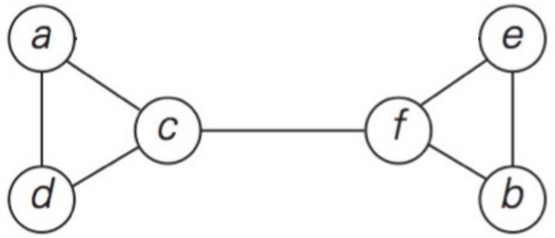


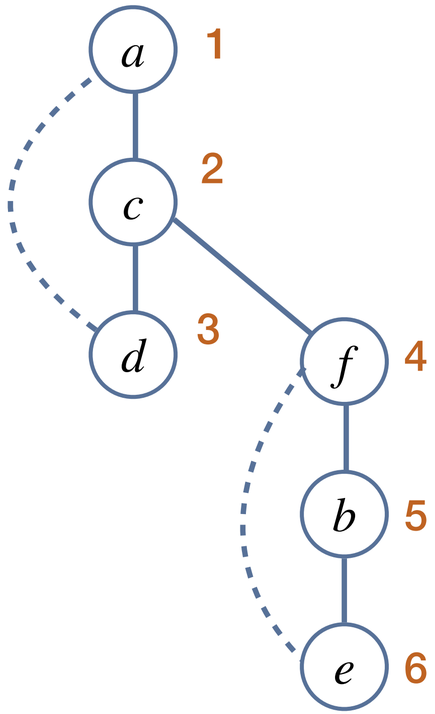
Is v is an AP ?

Finding Articulation Points - Tarjan's Approach

While constructing the DFS tree, keep track for each node v the following quantities:

- Parent of v
- Discover[v] = Visit order of v in the traversal.
- Lowest[v] = Lowest visit order of node that v and its proper descendants can reach via back edge, or Discover[v] otherwise.





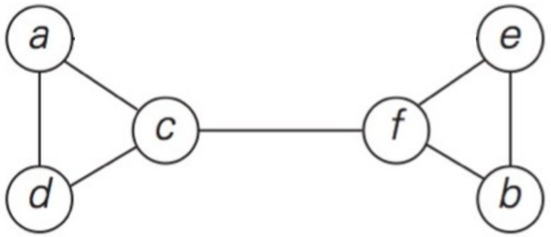
Node	Parent	Discover	Lowest
a			
c			
d			
f			
b			
e			

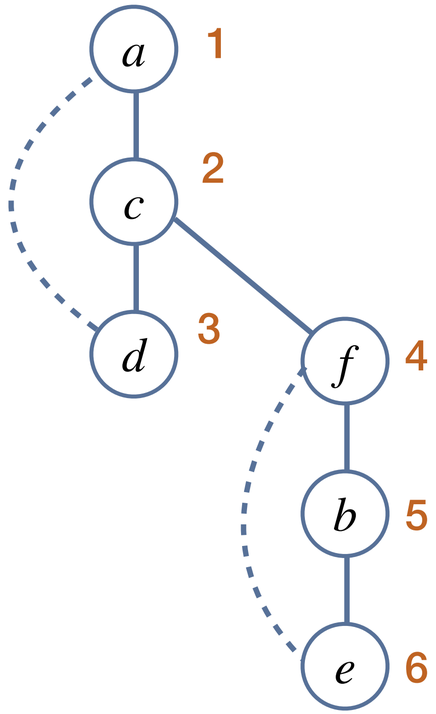
When performing DFS(v),

Initialize Lowest[v] and Discover[v] to the visit order.

If neighbor w is unvisited, run DFS(w) and Lowest[v] = min(Lowest[v], Lowest[w]).

Else If neighbor w is not the parent of v, Lowest[v] = min(Lowest[v], Discover[w]) (Back edge)





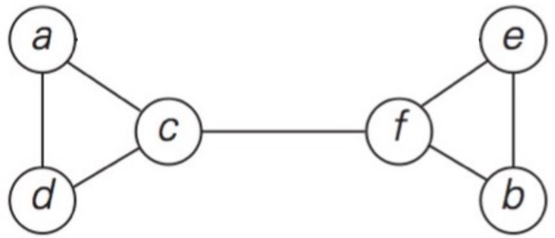
Node	Parent	Children	Discover	Lowest
a				
c				
d				
f				
b				
e				

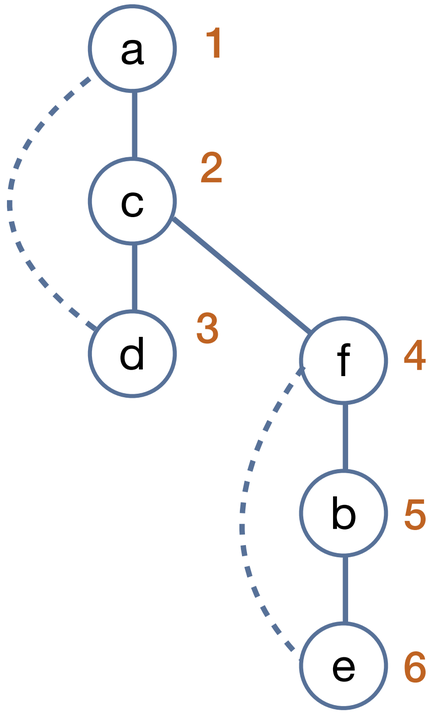
A ***root node*** v in DFS tree is an AP if it has at least two children.

An ***internal node*** v in DFS tree is an AP if it has a child w with Discover[v] $\leq$ Lowest[w].

Examples

- Is node a an articulation point ?
- Is node c an articulation point ?

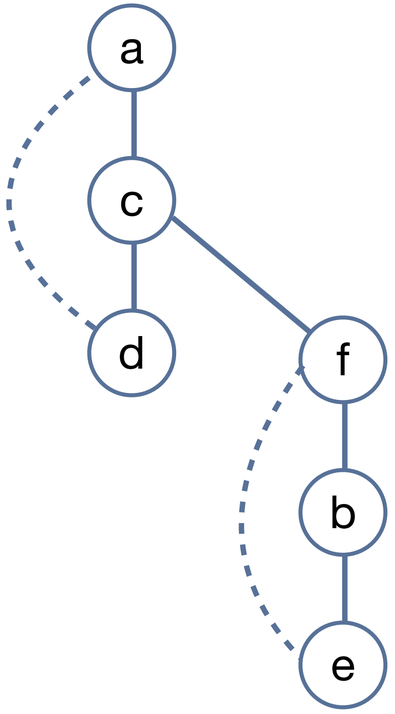


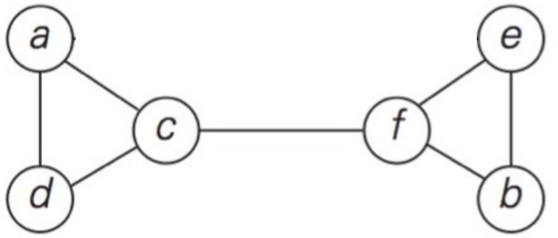


Node	Parent	Children	Discover	Lowest
a				
c				
d				
f				
b				
e				

Algorithm TarjanAP

```
1:  Input: Connected graph G = ⟨V, E⟩
2:  Output: Array isAP[] indicating if u is an AP.
3:
4:  Discover[v],Lowest[v] ← 0 ∀v ∈ V (being unvisited)
5:  numChildren[v] ← 0 ∀v ∈ V
6:  isAP[v] ← False ∀v ∈ V
7:  count ← 0
8:  for each vertex v in V do
9:      if v is marked with 0 then DFS(v)
10:
11: procedure DFS(v)
12:     count ← count+1
13:     Discover[v], Lowest[v] ← count,  Parent[v]  ← v
14:     for each vertex w adjacent to v do
15:         if Discover[w] == 0 then                    ▷ Unvisited neighbor w
16:             Parent[w] ← v
17:             numChildren[v] += 1
18:
19:             DFS(w)
20:             Lowest[v] ← min(Lowest[v], Lowest[w])
21:
22:             if Discover[v] == 1 and numChildren[v] ≥ 2 then isAP[v] ←True
23:             if Discover[v] > 1 and Lowest[w] ≥ Discover[v] then isAP[v] ←True
24:         else if w ≠ Parent[v] then                  ▷ v has a back edge to w
25:             Lowest[v] ← min(Lowest[v], Discover[w])
26:
```





Node	Discovery order	Lowest
a	1	1
c	2	1
d	3	1
f	4	4
b	5	4
e	6	4

Self Test: AP Algorithms



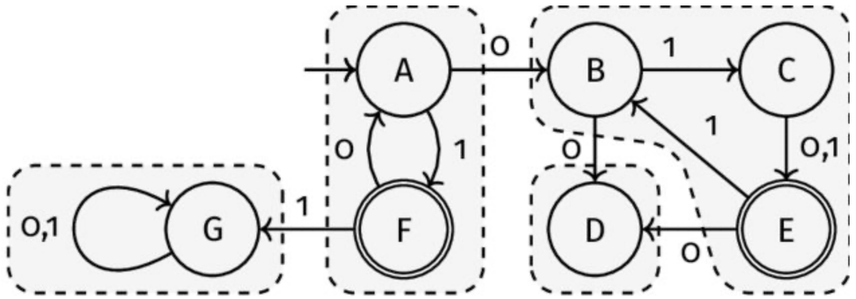
https://forms.cloud.microsoft/r/xFt0g2JuyV?origin=lprLink

Scenario: Urban Traffic Planning

City traffic engineers often convert two-way streets into one-way streets to streamline traffic flow and increase road capacity.

- However, poor design can accidentally create "dead zones".
- How to make sure that there exists no dead zones after the street conversions?

Does an FSM have a dead-lock zone besides the terminating one?





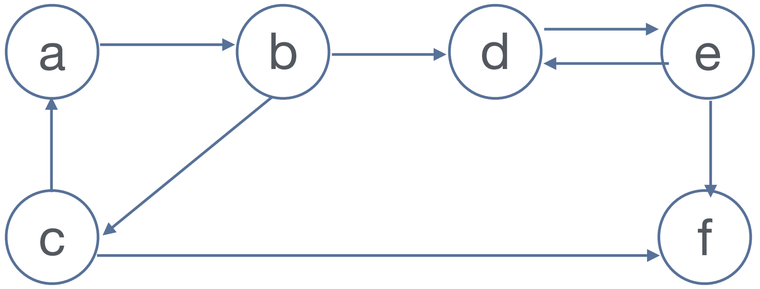
(a) variant A (b) variant B

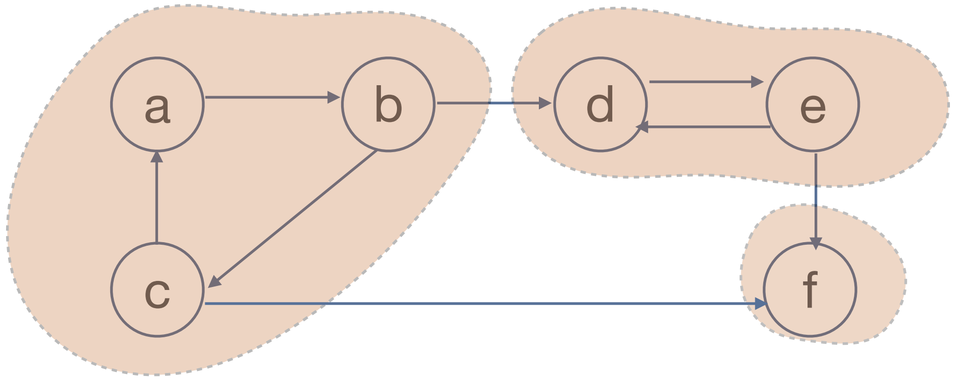


(c) variant C

Strongly Connected Components (SCC)

A ***strongly connected component (SCC)*** of a digraph is a maximal (largest) strongly connected subgraph.



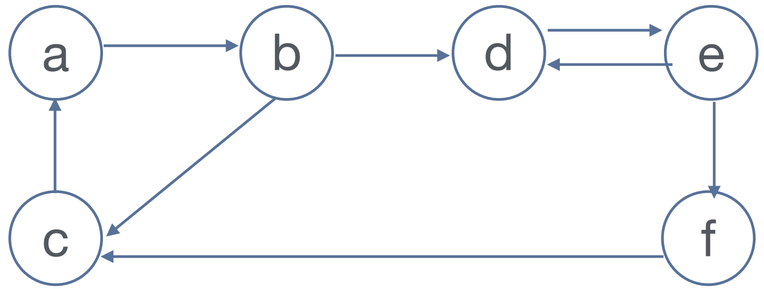


Acyclic component graph (3 SCCs)

Basic Idea

What happens if we run DFS on the original graph?

- May leave one SCC and enter another.
- DFS tree not necessarily SCC.

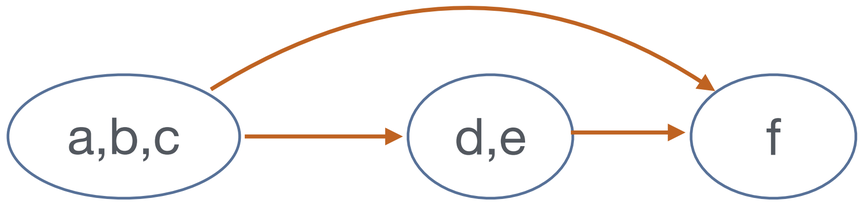


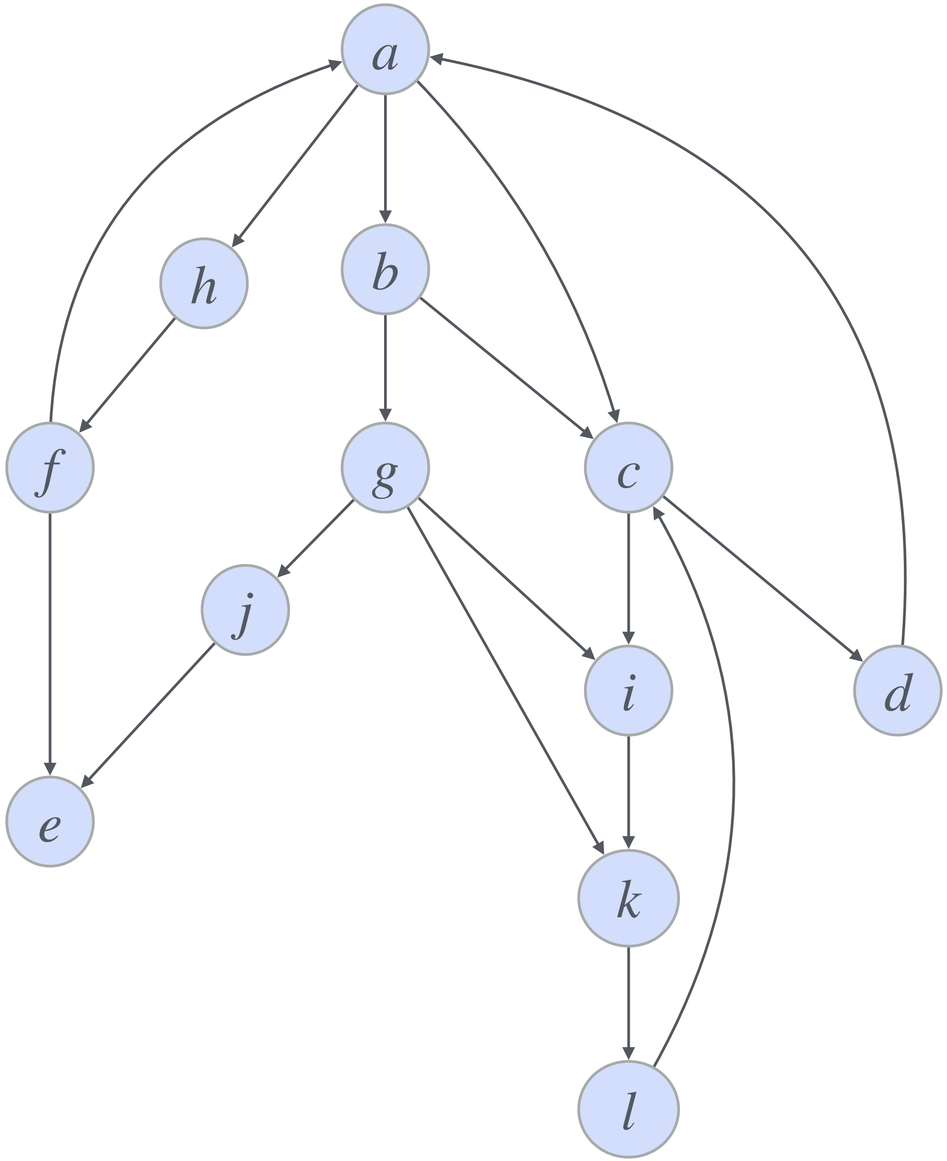
However, we can determine the correct order in which SCCs should be explored from DFS.

Note that SCC nodes must form a DAG.

If we reverse every edge in the original graph and run DFS with an appropriate node order.

- Nodes in SCC are still be strongly connected.
- Traversal in the current SCC cannot escape to another unprocessed SCC.



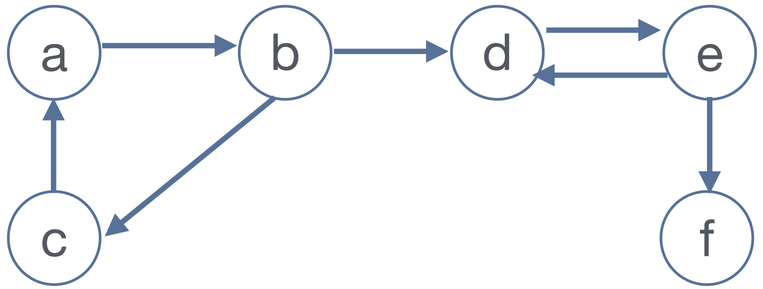


Visit (push) order:

Finish (pop) order:

Kosaraju's (Kosaraju–Sharir) Algorithm

Run DFS on the original graph and keep track of the finishing (popping) order of nodes.



$f_{5,1}$

$e_{4,2}$

$d_{3,3}$ $c_{6,4}$

$b_{2,5}$

$a_{1,6}$

Order nodes by finishing times (popping order): f, e, d, c, b, a

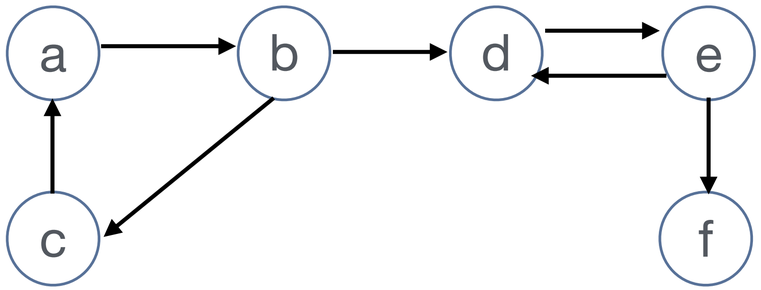
Order nodes by decreasing finishing times (reverse popping order): a, b, c, d, e, f

Run another DFS on the ***transposed graph*** G^T, starting with the node with ***largest*** finishing time v as root.

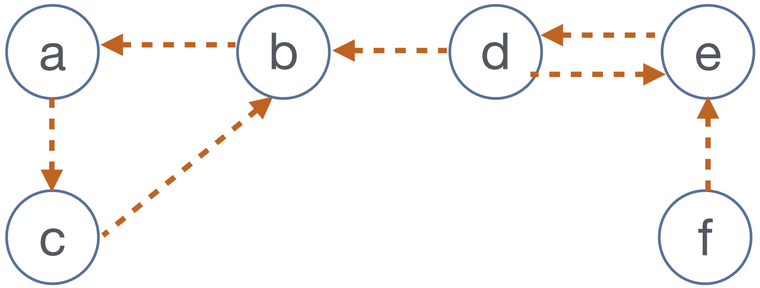
- Consider the nodes in the order of decreasing finishing time.
- For each node traversed, assign it to a connected component with root v
- Stop when each node has been assigned to some component.

One DFS tree = One SCC

Original graph



Transposed graph



Nodes ordered by decreasing finishing times: a, b, c, d, e, f

In-Class Assignment L4-1 (Group of 4 or less): Strongly-Connected Component

(20 minutes) Submit to LEB2.